

Your work, in your own data

The twelve openstory-skills, run against your live store. The same reports as the prompt library — but every number here is yours.

How this was made: each section is the real output of one of the twelve `openstory-skills` run against your live store over **the last ~30 days (arc & recall reach across all history)** — the same reports, charted from your own history. Names of collaborators are shown as roles; secrets are never included.

The twelve skills

Each one is a Claude Code slash command (the `openstory-skills` plugin). One per page that follows — with your real numbers.

01	<code>/cost</code>	what your sessions cost
03	<code>/tools</code>	what you reach for
05	<code>/arc</code>	the story so far
07	<code>/standup</code>	today
09	<code>/prime</code>	pick up where you left off
11	<code>/team</code>	who's working

02	<code>/time</code>	where your time goes
04	<code>/coach</code>	how you work
06	<code>/recap</code>	what shipped lately
08	<code>/recall</code>	find it again
10	<code>/watch</code>	live work
12	<code>/scan</code>	before you share

01 /cost · what your sessions cost

What have my agent sessions cost me?

6.46B

total tokens (30d)

~\$3.2K

est. cost (Sonnet pricing)

96%

cache hit rate

\$16.5K

saved vs. no-cache



A 96% cache hit rate turned a potential **\$19.8K** bill into **\$3.2K** — prompt caching saved ~\$16.5K over 30 days. (Sonnet pricing: \$3/M in, \$0.30/M cache read, \$15/M out.)

daily_token_usage (days=30)

02 /time · where your time goes

Where does my time actually go?

ACTIVITY BY HOUR (UTC, 4-HOUR BUCKETS)



1 AM UTC

peak hour (8,706 events)

27%

top-3 hours share

68.2K

total events (30d)

The dead zone is **UTC 04-07** (near-zero) — consistent with sleep. Three active windows: UTC 12-15, 20-23, and 00-03. Not a 9-to-5 pattern — work clusters into morning, afternoon, and a late-evening push.

productivity (days=30) — hourly event density

03 /tools · what you reach for

Which tools and commands do I rely on most?

TOP TOOLS (8 SESSIONS, ~14 DAYS)



3,893

total tool calls

2.3:1

Edit : Read ratio

59%

Bash share

Bash dominates at 59% of visible calls — shell execution is the primary agent action. Edit outpaces Read 2.3:1: a write-heavy pattern, the agent codes more than it explores.

list_sessions(days=14) + session_synopsis on the 8 largest sessions

04 /coach · how you work

Honest feedback on how I work with coding agents.

SIGNAL	RATE	MEANING
Multi-day sessions	High	Sessions run 4–7 days each; context degrades and errors concentrate late in the run
Bash dominance	50% of tools	Execution-first: ops, greps, curls dominate structured reads and edits
Edit-before-Read ratio	Medium	1.85:1 — editing nearly 2x as often as reading; risk of edits without full context
Error events	Medium	23 errors in one 7-day session; the other 3 were clean — pattern is session-age correlated
Agent delegation	2% of tools	Subagents used rarely — research and exploration stay serial
Compaction hits	Low	1 compact across ~7 days; context rarely hits the hard limit
#1 pattern: sessions run multi-day (4–7 days each). That is where errors appear and context quality silently drifts. Fix: close and restart a session when a concrete phase ends — don't carry one thread across days of varied work.		
4 recent OpenStory sessions Jun 17–26 — 1130 tool calls total		

05 /arc · the story so far

Trace the story of how this project came to be.

- **2026-06-14**
CI/CD deployment pipeline
Enable automatic CI/CD — 1,194 events, Bash-heavy; infra pipeline hardening
- **2026-06-16**
Strategic reflection
Deep reflection on where the work stands — direction-setting, 2,112 events
- **2026-06-17**
Prompt-library generators
Analysed most-common prompts → built the generators + 5-palette theme system; 2,752 events
- **2026-06-20**
Developer-archetype research
Multi-agent adversarial fact-checking — extractor + verifier subagents; archetype charts built
- **2026-06-21**
Architectural audit + hub connection
Full architectural assessment (1,694 events) + NATS hub connectivity in parallel
- **2026-06-22**
Cross-agent integration debug
Getting remote-agent events into OpenStory; install troubleshooting; CLAUDE.md audit
- **2026-06-26**
Team skill + UI wireframes
openstory:team automation run; UI review; wireframe design; codebase scan

Two weeks of parallel construction — CI/CD hardening, strategic reflection, prompt-library and archetype research, a deep architectural audit, cross-agent integration, and team-skill automation: OpenStory building itself while building the tooling to observe everything else.

`list_sessions(project=OpenStory)` — 200 sessions, Jun 14 → Jun 26

06 /recap · what shipped lately

What have I worked on lately, grouped by project?

PROJECT	SESSIONS	ACTIVITY		FOCUS
OpenStory	243	events	15,694	Prompt-library generators, archetype research, UI review, NATS federation, team-skill automation
workspace	3	events	6,848	Remote agent hub sessions (OpenClaw check-ins + hub connectivity)
agent-harness	4	events	5,399	OpenActor architecture, reactive event-driven system planning
financial-research	1	events	3,146	Financial / economic research and policy analysis
openstory-deploy	1	events	1,194	CI/CD deployment pipeline automation
openstory-website	1	events	206	Website updates
OpenStory leads at 15k+ events across 243 sessions — including ~200 short script-layer agent sessions spawned by the prompt-library and archetype generators. Three parallel fronts: OpenStory feature work, remote agent sessions via OpenClaw (workspace), and OpenActor reactive design (agent-harness).				
project_pulse(days=14) merged by display name + list_sessions(days=14)				

07 /standup · today

Write my standup: what I did, what's blocked, what's next.

DID	BLOCKED	NEXT
• Ran `openstory:team` skill — 583 events, ~3.3h; heavy Bash + Task automation (Jun 26) • Ran `openstory:scan` — codebase scan, 41 events (Jun 26) • UI review + design discussion — assessed UI quality and direction (Jun 26) • Wireframe exploration for a new feature — subagent design session (Jun 26) • Two PR-review sessions — code walkthrough + review (Jun 25)	• NATS leaf federation — 122-event investigation (Jun 25) into why the leaf node isn't reaching the hub; no resolution recorded	• Resolve NATS leaf federation — continue from the Jun 25 investigation • Iterate on UI design from the wireframe exploration + review feedback • Follow up on openstory:team skill findings
Last active Jun 26. 4 sessions / 672 events that day; 3 more (294 events) on Jun 25. Top tools: Bash, TaskUpdate, Edit — a mix of skill automation, UI review, design, and PR analysis.		
<code>list_sessions(days=7) + session_synopsis</code> for the top sessions on Jun 25-26		

08 /recall · find it again

Find the last time I solved this — and exactly how.

Jun 25	Make sure we are connected to the hub	OpenStory (remote)
Jun 21	Architectural assessment — touched federation config	OpenStory
older	NATS leaf bootstrap + deploy/ script	OpenStory
older	Tailnet federation spike — hub config	OpenStory
older	NATS leaf secrets + .env.federation authoring	OpenStory

1

curl -s http://localhost:8222/varz | python3 -c "import sys,json; d=json.load(sys.stdin); print('leafnodes:', d.get('leafnodes'))"

2

grep -inE 'leaf|remotes|7422|url|hub' nats.conf

3

cat .env.federation # NATS_LEAF_URL=nats://<redacted>@<redacted>:7422

4

pgkill -f 'nats-server -c nats.conf' # free :4222 so the leaf node can bind it

5

nohub just up-federation > /tmp/openstory-federation-boot.log 2>&1 &

6

curl -s http://localhost:8222/leafz # out->hub / in<-hub message counts confirm the link

7

curl -s http://localhost:3002/api/sessions # both hosts' sessions should now appear

Topic recalled: NATS leaf federation — connecting local OpenStory to the distributed hub over Tailscale. Last solved Jun 25, 2026. Key insight: a standalone `nats.conf` occupies `:4222` and blocks the leaf node; `up-federation` sources `.env.federation` and starts the leaf instead.

agent_search 'federation nats leaf deploy' - 5 matches; tool_journey on the top session (Jun 21-25)

09 /prime · pick up where you left off

Where did we leave off on this project?

LAST SESSION	Jun 26, 2026 04:06 UTC (openstory:team)
LAST GOAL	Team-day planning + UI wireframe design for a new feature
OPEN THREADS	prompt-library-generators branch; UI redesign + watch-command-center research; developer-archetype profiling
TOP TOOLS	Bash 69 · TaskUpdate 13 · Edit 10 · TaskCreate 9

Prompt-library generators branch: tooling/prompt-library-generators
Theme system (5 palettes) + session-usage analysis built. Several scripts still untracked — stage and PR.

Developer-archetype research docs/research/developer-archetypes/
Citation-traceable archetype profiling with viz scripts. Style axis refined to process-locus. Docs and scripts untracked.

UI redesign + watch command center docs/research/ui-redesign/
Wireframe session ran Jun 26; UI review completed same day. Feature research docs untracked.

Boot skill .claude/skills/boot/
New boot skill formalizing the full boot flow (Docker / native SQLite / split dev) — untracked.

Suggested next step: **stage and open a PR for the prompt-library-generators branch** — most new work (scripts/, docs/research/, the boot skill) is still untracked.

project_context for OpenStory (last 5 sessions Jun 25–26) + git status on the active branch

10 `/watch` · live work

Watch the work happening right now and summarise it.

04:06	Bash	Check UI build log — dist dir present after background npm run build
03:47	Read	Read rs/cli/src/main.rs — inspect --static-dir serve flag
03:46	Bash	Search build_router + static serve wiring across rs/ crates
03:46	Bash	Launch UI production build in background, check vite proxy config
03:45	Bash	Inspect CLI serve subcommand args and vite.config.ts proxy settings
03:41	AskUserQuestion	Ask: serve new WatchBanner build via --static-dir or keep Vite proxy?
03:41	Bash	Verify open-story serve + Vite processes on :3002 / :5173
03:40	Bash	Test /api/sessions response count + WS upgrade path live

Static snapshot of session **d58d3d03** (started 00:46 UTC, Jun 26). In the live deck this feed streams in real time from the ``/ws`` WebSocket. This session built the watch-focus feature — ``POST /api/watch/{id}`` + ``WatchBanner`` UI — the same mechanism ``/watch`` uses to aim the UI at a session.

tool_journey for most recent session d58d3d03 (last_event 2026-06-26T04:06 UTC, 583 events)

11 `/team` · who's working

Who on my team is active, and on what?

You · local 29 sessions

OpenStory core work: agent-directed watch-focus endpoint (POST `/api/watch` + WatchBanner UI), PR reviews, CI/CD deploy automation, prompt-library generators, `openstory:team/scan/coach` skills, CLAUDE.md audit, NATS federation debugging

You · remote 9 sessions

Remote Linux box: NATS leaf node federation, OpenActor/agent-harness development, cross-machine OpenStory connectivity, economic research one-off

OpenClaw agent 3 sessions

Automated workspace reads (HEARTBEAT.md context pattern on startup); long-running session spanning months with periodic activation from the VPS

OpenActor (automated) 1 session

Single automated benchmark run (coding QA task, short-lived)

This is a **single-operator store** — all sessions belong to you across two machines plus two agents. `/team` lights up with real multi-user data when a teammate's NATS leaf node federates in: their sessions arrive with a distinct host prefix and appear as a separate card here automatically.

```
list_sessions days=7 (56 sessions); project_id path prefixes identify 4 distinct host/agent segments
```

12 /scan • before you share

Is there anything sensitive in my history?

CATEGORY	HITS	STATUS
`sk-` API key prefix	0	Low
`ghp_` GitHub token prefix	0	Low
`AKIA` AWS key prefix	0	Low
`BEGIN PRIVATE KEY`	0	Low
`xoxb` / `xoxp` Slack token	0	Low
`password` keyword	10	Medium
`Bearer` / `api_token`	0	Low

PATTERNS SCANNED

sk-ghp_AKIABEGIN PRIVATE KEYxoxbxoxppasswordBearerapi_token

All high-signal key prefixes are **zero** — confirmed clean. The 10 ``password`` hits are session-transcript context only: SSH login prompts from a server setup and dummy test-fixture credentials in a NATS multi-account test config — no real credentials. ``Bearer`/`api_token`` hits are Rust middleware source snippets read during code review, not live token values.

FTS5 search across live event store for 7 sensitive-pattern categories; baseline confirmed by dedicated /scan session (80b66f5f) which ran grep against the transcript tree